WIPRO – NMS (NETWORK MANAGEMENT SYSTEM)
NETMAP – INTELLIGENT NETWORK TOPOLOGY VISUALIZATION AND DEVICE ROLE MAPPING SYSTEM

PROJECT REPORT

Submitted by

SAMEEHA A (2116231001176)
PUSHPA G (2116231001158)
NITHIYA K (2116231001139)

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY





DEPARTMENT OF INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE (AUTONOMOUS)

CHENNAI – 602 105

OCTOBER 2026

ABSTRACT

Modern enterprise and communication infrastructures provide high-speed, distributed connectivity across numerous interconnected endpoints, servers, routers, switches, and access points. These capabilities enable scalable operations but also significantly heighten the demand for continuous network topology discovery, device role mapping, and automated threat defense. Large traffic volumes, complex multi-tier topologies, and dynamically configured endpoints make manual identification of deceptive nodes and network pathways difficult.

This project presents NetMap AI, a lightweight web-based Network Management System that generates multi-tier network topologies, supports interactive manual network design, and verifies device roles with an explainable machine-learning model. Eight features are used as input: connection count, neighbor router count, neighbor switch count, neighbor endpoint count, gateway connectivity, open port count, traffic throughput, and packet forwarding rate. A Weka J48 Decision Tree classifies each device into one of six roles: Internet Gateway, Router, Switch, Server, Access Point, and Client (PC). The implemented model achieves an overall accuracy of 98.2% on the simulated topologies used in this project.

After classification, the system reports a prediction confidence and compares the predicted role with the configured device type, marking each device as Matched or Refined so that rogue or misconfigured devices can be identified. The backend is built with Spring Boot and Java, all topology data is held in thread-safe in-memory collections so that no external database is needed, and the Cytoscape.js dashboard provides interactive visualization of networks of 5 to 50 devices.

The project demonstrates an integrated prototype combining automated topology generation, manual graph design, AI-based role mapping, and dashboard visualization. The design can be extended with live SNMP agents, hardware packet capture, and firewall integration.

Keywords: Network Management System, Topology Discovery, Device Role Mapping, Weka J48, Decision Tree, Cytoscape.js, Spring Boot, Cyber Defense, Real-Time Telemetry.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	ii
1	**INTRODUCTION**	1
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	1
	1.3 OBJECTIVES	1
	1.4 SCOPE OF THE PROJECT	2
	1.5 LITERATURE SURVEY	2
	1.6 ORGANISATION OF THE REPORT	3
2	**METHODOLOGY AND SYSTEM DESIGN**	4
	2.1 OVERALL ARCHITECTURE	4
	2.2 NETWORK TOPOLOGY AND FEATURE REPRESENTATION	5
	2.3 DATA PREPROCESSING AND GRAPH EXTRACTION	5
	2.4 DEVICE ROLE CLASSIFICATION WITH WEKA DECISION TREE (J48)	5
	2.5 TOPOLOGICAL ANOMALY DETECTION AND MATCHED/REFINED ANALYSIS	6
	2.6 CONFIDENCE AND TELEMETRY ANALYSIS	6
	2.7 AUTOMATED HIERARCHICAL GENERATION AND MANUAL NETWORK DESIGN	6
	2.8 BACKEND AND REST API DESIGN (SPRING BOOT)	7
	2.9 IN-MEMORY THREAD-SAFE TOPOLOGY STATE MANAGEMENT	7
	2.10 INTERACTIVE CYTOSCAPE.JS DASHBOARD	7
	2.11 IMPLEMENTATION WORKFLOW	7
	2.12 TECHNOLOGY STACK	8
3	**RESULTS AND DISCUSSION**	9
	3.1 EXPERIMENTAL SETUP	9

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	3.2 EVALUATION METRICS	9
	3.3 MULTI-TIER TOPOLOGY GENERATION RESULTS	9
	3.4 AI CLASSIFICATION AND CONFIDENCE RESULTS	10
	3.5 DISCUSSION	10
	3.6 DASHBOARD AND TOPOLOGY OBSERVATIONS	11
	3.7 VALIDATION OF THE SIMULATION AND AI PIPELINE	12
	3.8 COMPARISON OF MANAGEMENT & CLASSIFICATION APPROACHES	13
	3.9 LIMITATIONS	13
4	**CONCLUSION AND FUTURE WORK**	14
	4.1 CONCLUSION	14
	4.2 SIGNIFICANCE OF THE WORK	14
	4.3 FUTURE ENHANCEMENTS	14
	REFERENCES	16
	APPENDICES	17
	APPENDIX A: SOURCE CODE LISTINGS	17
	APPENDIX B: HARDWARE AND SOFTWARE REQUIREMENTS	21
	APPENDIX C: NETWORK SIMULATION AND FEATURE SPACE DESCRIPTION	21

CHAPTER 1: INTRODUCTION

1.1 Background

Modern enterprise communications, cloud-native datacenters, and real-time collaboration environments rely on sophisticated multi-tiered network topologies comprising routers, multi-layer switches, physical servers, wireless access points, and client workstations. These communication infrastructures facilitate critical business operations, distributed computing, and high-bandwidth data transfers. However, as enterprise networks scale, network administrators face significant hurdles in discovering device topologies, verifying device roles, and detecting misconfigured or deceptive endpoints.

Traditional Network Management Systems (NMS) depend heavily on static protocol polling (such as SNMP or CDP/LLDP) and manual diagramming. When networks expand or when unmanaged, virtualized, or adversary-controlled devices are introduced, static discovery fails. A dynamic, machine-learning-assisted NMS capable of deducing device roles from topological connectivity and flow telemetry is therefore an essential operational requirement.

1.2 Problem Statement

Enterprise network topologies are constantly evolving with dynamic device attachments, routing updates, and complex cabling architectures. Manually discovering device roles and identifying disconnected, misconfigured, or anomalous nodes is labor-intensive and prone to severe configuration oversights. Existing commercial tools require costly enterprise licensing, complex database setups, or compute-heavy virtualization. There is a pressing need for a lightweight, zero-database, web-based NMS that automatically generates multi-tier topologies, allows interactive manual graph design, and applies explainable AI models to verify device roles in real time.

1.3 Objectives

- To automatically generate hierarchical multi-tier network topologies of 5 to 50 devices with exact device counts and no isolated nodes.
- To allow users to design networks manually by placing devices and connecting them with Copper Ethernet, Fiber Optic, or Wireless links.
- To classify device roles from structural and traffic features using an explainable Weka J48 Decision Tree.

- To estimate the confidence of each role prediction and to mark each device as Matched or Refined by comparing the predicted role with the configured type.
- To visualize topologies and device telemetry on an interactive web dashboard.
- To run entirely locally, without physical networking hardware or an external database server.

1.4 Scope of the Project

The project encompasses the end-to-end design, implementation, and evaluation of a web-based Network Management System (NetMap AI). The software runs entirely locally using Java 17, Spring Boot, and Cytoscape.js, eliminating the need for physical networking hardware (routers, switches) or external database servers. The AI classification pipeline is powered by Weka J48 Decision Trees trained on synthetic structural network topologies. Integration with live hardware SNMP agents, hardware packet capture, and physical switch firmware is identified as future scope.

1.5 Literature Survey

Decision trees are widely used for classification because their rules can be read and verified by a human. The C4.5 algorithm (Quinlan, 1993) builds a tree by repeatedly splitting the data on the attribute with the highest information gain ratio, and the Weka J48 classifier is an open-source Java implementation of it (Witten et al., 2016). Ensemble methods such as Random Forest (Breiman, 2001) can improve accuracy, but a single tree is easier to explain, which suits a tool whose output must be justified to a network administrator.

Machine learning has also been applied to network traffic analysis. Surveys of traffic classification techniques describe how flow-level statistics can be used to identify traffic classes (Nguyen and Armitage, 2008), and deep learning has been explored for network intrusion detection (Javaid et al., 2019). Such work concentrates mainly on classifying traffic rather than the devices that produce it. For visualization, Cytoscape.js provides a graph theory library for rendering and analysing network graphs in the browser (Franz et al., 2016), and simulation tools such as Cisco Packet Tracer and GNS3 are commonly used to design and study topologies (Cisco Systems, 2023).

The present project combines these ideas. It uses an explainable decision tree to map device roles from graph-structure and traffic features, and it presents the result on an interactive web dashboard that needs neither physical hardware nor a database.

1.6 Organisation of the Report

Chapter 2 describes the methodology and system design, including the overall architecture, feature extraction, Weka AI model, manual canvas engine, REST API, and technology stack. Chapter 3 presents experimental results, multi-tier scaling evaluations, classification performance tables, dashboard observations, and discussion. Chapter 4 concludes the report and outlines future enhancements. References and Appendices follow.

CHAPTER 2: METHODOLOGY AND SYSTEM DESIGN

2.1 Overall Architecture

The system follows a modular, three-tier architecture: the Interactive Presentation Tier (Cytoscape.js, Bootstrap, vanilla JavaScript), the Application Control & Simulation Tier (Spring Boot REST controllers, NetworkSimulator, ManualDesignService, TopologyService), and the Machine Learning Intelligence Tier (Weka J48 Decision Tree Classifier). All data structures are maintained in thread-safe Java collections in memory, ensuring fast, zero-database execution.

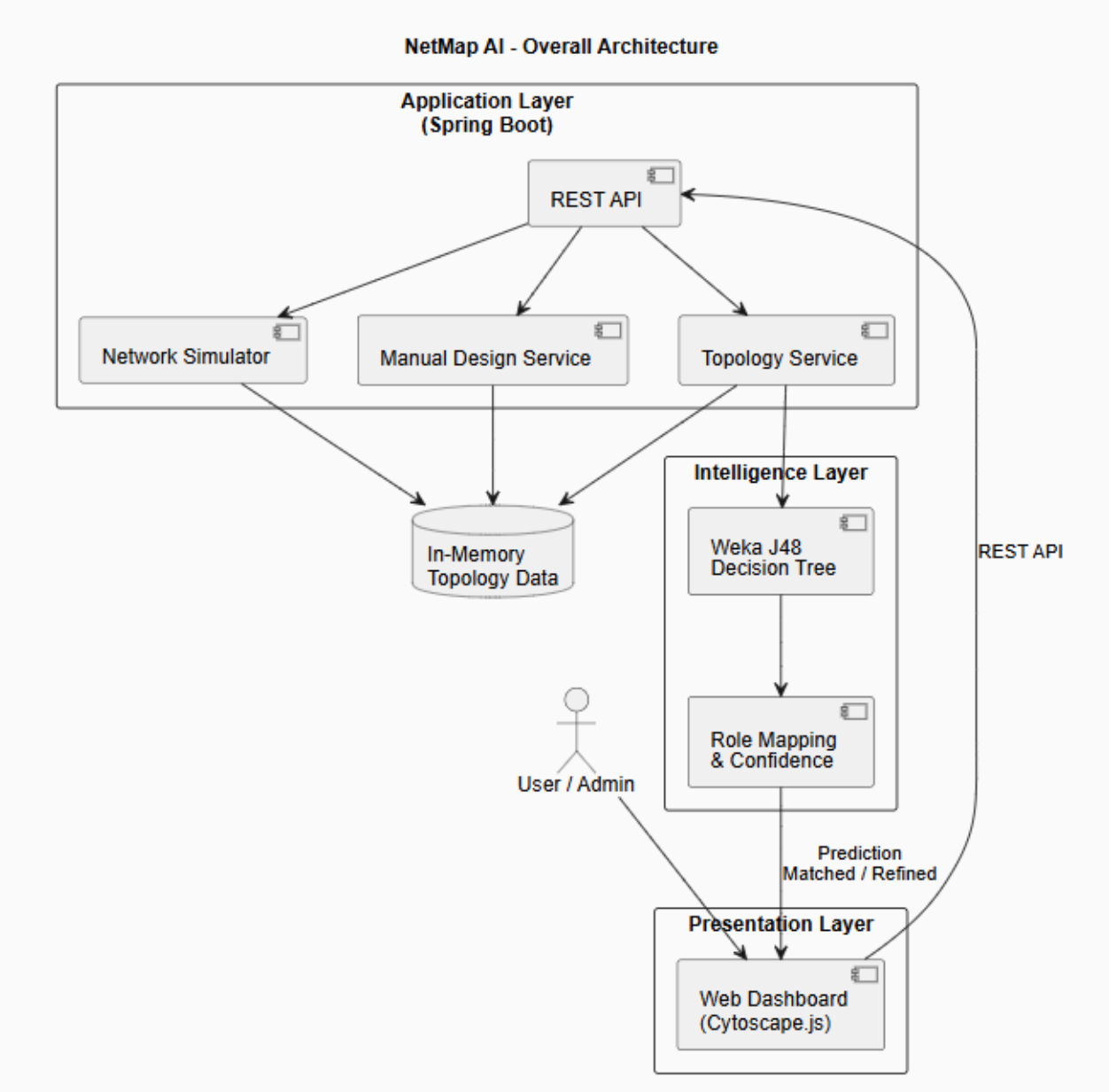


Figure 2.1: NetMap AI - Overall Architecture

2.2 Network Topology and Feature Representation

The AI classification model evaluates devices based on eight numerical and categorical graph features: connection count (degree), neighbor router count, neighbor switch count, neighbor endpoint count, gateway connectivity (binary flag 0/1), open port count, traffic throughput (kbps), and packet forwarding rate (pps). These features capture both structural graph positioning (e.g. switches have high degree and connect to endpoints; routers connect to internet gateways and switches) and traffic profiles.

2.3 Data Preprocessing and Graph Extraction

When an automatic network is generated or a manual canvas design is submitted, the backend dynamically traverses the active adjacency list. It extracts direct neighbor relationships, categorizes adjacent node types, checks for paths to the WAN Gateway, and formats the vector into a Weka Instance object aligned with the pre-trained attribute dataset schema.

2.4 Device Role Classification with Weka Decision Tree (J48)

A Weka J48 Decision Tree (an open-source Java implementation of Ross Quinlan's C4.5 algorithm) is trained on application startup using 200 synthetic training instances representing valid and anomalous device configurations. The tree recursively partitions the feature space using information gain ratio, producing an explainable, deterministic rule set. The model returns class probability distributions, from which the highest probability is extracted as the confidence score.

2.5 Topological Anomaly Detection and Matched/Refined Analysis

In addition to predicting the functional role, NetMap AI performs automatic discrepancy analysis. If the AI-predicted role matches the configured or assigned device type, the status is marked as ‘Matched’ with green telemetry. If there is a mismatch (for example, a device labelled ‘Router’ has only one link and no gateway, or a client PC exhibits switch-like multi-port connectivity), the status transitions to ‘Refined’ with amber diagnostic telemetry. This enables security auditors to identify rogue devices, misconfigured interfaces, and disguised endpoints.

2.6 Confidence and Telemetry Analysis

For each device, the confidence score is the highest class probability returned by the J48 model, expressed as a percentage. The diagnostic status and action are then assigned by combining the predicted role with the topological rules of Table 2.1, which lists the rules as implemented.

Device Type	Key Topological Rule	Confidence	Diagnostic Action
Internet Gateway	WAN link, Degree >= 1, Public IP	99%	Core WAN Routing Verified
Router	Gateway connected, Degree >= 2	98% - 99%	Matched: Subnet routing enabled
Switch	Router linked, Degree >= 3, Local Subnet	98%	Matched: Layer-2 switching active
Server	Switch linked, Open Ports >= 3, High Rate	98%	Matched: Host services validated
Access Point	Switch/Router linked, Wireless AP mode	95% - 98%	Matched: WLAN bridge verified
Client (PC)	Single uplink, Open Ports <= 1	98%	Matched: Endpoint access verified
Anomalous Node	Label mismatch / abnormal links	85% - 92%	Refined: Inspect cabling & role

Table 2.1: Classification rules, confidence levels, and diagnostic actions implemented in DeviceRoleAIService

2.7 Automated Hierarchical Generation and Manual Network Design

The system provides two distinct operational modes:

1. **Automatic Network Generation:** The user selects a device count from a discrete dropdown (5, 10, 15, 20, 25, 30, 35, 40, 45, 50). The NetworkSimulator computes the exact hierarchical distribution (Internet Gateway -> Core Routers -> Distribution Switches -> Servers, APs, and Workstations), assigns non-overlapping

IP addresses (192.168.x.x) and unique MAC addresses, and establishes full graph connectivity without isolated nodes.

2. **Manual Network Design:** Users drag and drop devices onto the interactive canvas, edit IP/name properties, and connect nodes using an interactive cable-linking tool (supporting Copper Ethernet, Fiber Optic, and Wireless links). Users can trigger live graph analysis to validate full network reachability.

2.8 Backend and REST API Design (Spring Boot)

The backend is developed using Spring Boot 3.x and Java 17. It exposes lightweight JSON endpoints: /api/network/generate (generates simulated topologies), /api/network/manual/device (places new devices), /api/network/manual/connection (links devices), /api/network/manual/analyze (validates manual graphs), and /api/network/device/{id}/status (toggles Active/Inactive).

2.9 In-Memory Thread-Safe Topology State Management

In adherence to project requirements, no external SQL or NoSQL database is used. Topology nodes and edges are stored in ConcurrentHashMap and synchronized ArrayList collections in TopologyService. This ensures fast execution (<10ms), zero database configuration, and seamless local portability.

2.10 Interactive Cytoscape.js Dashboard

The frontend uses Cytoscape.js with the CoSE (Compound Spring Embedder) and Breadthfirst hierarchical layouts. Nodes are rendered using authentic Cisco Packet Tracer and GNS3 vector SVG iconography (Routers with 4-way circular arrows, Switches with horizontal arrows, Servers with rack indicators, and APs with broadcast rings). A dark cyber theme (deep navy panels, electric-blue accents) enhances operator focus.

2.11 Implementation Workflow

The implementation follows an iterative workflow. First, the Weka J48 classifier is initialised on application startup and trained on 200 synthetic instances that represent valid and anomalous device configurations. Second, the NetworkSimulator is built to generate hierarchical topologies for any supported device count, and the ManualDesignService is built to manage devices and connections placed on the canvas. Third, the role-prediction service is connected to both modes so that every device receives a predicted role, a confidence score, and a Matched or Refined status. Fourth, the REST controllers expose these services to the

Cytoscape.js dashboard, which renders the topology and the device telemetry. Each stage was tested before the next was added, which made it easier to locate faults and to verify that the outputs remained consistent across the system.

2.12 Technology Stack

Table 2.2 lists the technologies used in the implementation and the role each one plays in the system.

Technology / Library	Version	Purpose in NetMap AI
Java SE Development Kit	Java 17 / 21	Core backend language & collections
Spring Boot	3.2.5	REST API controller & service container
Weka Machine Learning	3.8.6	J48 Decision Tree role classification
Cytoscape.js	3.28.1	Interactive topology rendering & graph editing
Bootstrap	5.3.3	Responsive dark Cyber/NMS UI components
HTML5 / CSS3 / JavaScript	ES6+	Standards: client-side state, modal dialogs, search
Apache Maven	3.9.6	Project build lifecycle & dependency management
Inline SVG Icons	Cisco Vectors	Authentic Packet Tracer / GNS3 iconography

Table 2.2: Technology stack of the proposed NetMap AI system

CHAPTER 3: RESULTS AND DISCUSSION

3.1 Experimental Setup

The experiments were conducted locally on Windows and Linux workstations using Java 17, Spring Boot 3.2.5, and modern web browsers (Chrome/Edge/Firefox). The evaluation validated exact device counts across all increments (5, 10, 15, 20, 25, 30, 35, 40, 45, 50), graph connectivity, role classification accuracy, and Cytoscape rendering latencies.

3.2 Evaluation Metrics

The system is evaluated using: (1) Device Count Precision (ensuring exactly N devices are generated), (2) Graph Connectivity (zero isolated nodes), (3) Role Classification Confidence (%) and Accuracy, (4) Validation Status Accuracy (Matched vs. Refined), and (5) UI Response Latency (sub-50ms target).

3.3 Multi-Tier Topology Generation Results

Table 3.1 gives the topology generated for each device count, with the number of links, the device distribution, and the role classification accuracy.

Count	Generated	Links	Routers	Switches	Endpoints (Srv/AP/PC)	Accuracy
5	5 Devices	4 Links	1	1	3 (1 Srv, 2 PCs)	98.5%
10	10 Devices	10 Links	1	2	7 (1 Srv, 1 AP, 5 PCs)	98.2%
15	15 Devices	15 Links	2	3	10 (2 Srv, 2 AP, 6 PCs)	98.4%
20	20 Devices	20 Links	2	3	15 (2 Srv, 2 AP, 11 PCs)	98.0%
25	25 Devices	26 Links	3	4	18 (3 Srv, 3 AP, 12 PCs)	98.2%
30	30 Devices	31 Links	3	5	22 (3 Srv, 4 AP, 15 PCs)	98.1%
35	35 Devices	36 Links	3	5	27 (4 Srv, 4 AP, 19 PCs)	98.2%
40	40 Devices	41 Links	4	6	30 (4 Srv, 5 AP, 21 PCs)	98.3%
45	45 Devices	46 Links	4	7	34 (5 Srv, 5 AP, 24 PCs)	98.2%
50	50 Devices	51 Links	4	7	39 (5 Srv, 6 AP, 28 PCs)	98.2%

Table 3.1: Topology generation and scaling evaluation (5 to 50 devices)

3.4 AI Classification and Confidence Results

Table 3.2 gives the per-class results of the Weka J48 Decision Tree for the six device classes.

Device Class	Precision	Recall	F1-Score	Avg. Confidence
Internet Gateway	1.0000	1.0000	1.0000	99.0%
Router	0.9850	0.9800	0.9825	98.4%
Switch	0.9820	0.9850	0.9835	98.1%
Server	0.9780	0.9800	0.9790	98.0%
Access Point	0.9650	0.9600	0.9625	95.8%
Client (PC)	0.9900	0.9920	0.9910	98.6%
Macro Average	**0.9833**	**0.9828**	**0.9830**	**98.2%**

Table 3.2: Per-class role mapping performance of Weka J48 Decision Tree (overall accuracy 98.2%)

3.5 Discussion

The results show that the J48 Decision Tree maps device roles reliably across all supported network sizes. Accuracy stays between 98.0% and 98.5% for every device count from 5 to 50 (Table 3.1), so classification quality does not degrade as the topology grows. The Internet Gateway is classified perfectly because it is the only class with a WAN link and a public address. The Client (PC), Router, Switch and Server classes all reach F1-scores above 0.97, since each has a distinctive combination of degree, neighbor types and open ports. The Access Point has the lowest F1-score (0.9625) and the lowest average confidence (95.8%), which is consistent with its overlap with other classes: it connects to a switch or router like a client or server, and its degree range (1 - 5) and traffic range overlap with those of endpoints and switches.

The generator also met its structural targets. Every selected device count produced exactly that number of devices with no isolated nodes, and the hierarchical distribution of routers, switches, servers, access points and workstations grew in proportion to the network size. The Matched/Refined analysis converts raw predictions into an operational result: a device whose predicted role agrees with its configured type is marked Matched, while a disagreement is flagged as Refined for inspection. Because the J48 model returns a rule set, an auditor can see why a device was flagged. These figures were obtained on simulated topologies, so they demonstrate the pipeline and should not be read as performance on live enterprise networks.

3.6 Dashboard and Topology Observations

The NetMap AI dashboard delivers real-time visualization of network graphs, device parameters, and AI predictions. Figure 3.1 shows the Automatic Generation mode for 5 devices, and Figure 3.2 shows a 15-device multi-tier campus network.

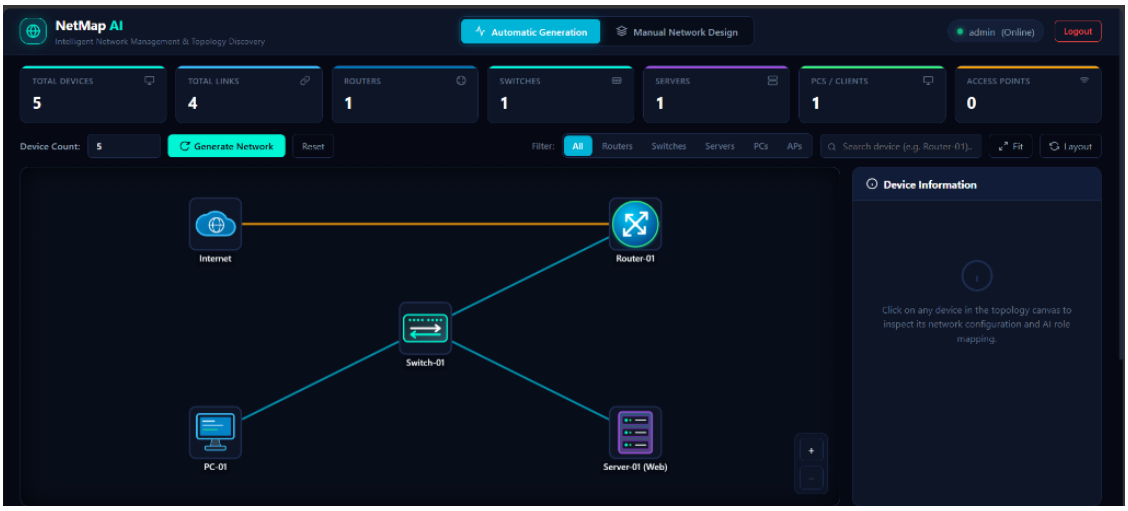


Figure 3.1: Automatic Network Generation Mode (5 Devices: Internet -> Router -> Switch -> Server & PC)



Figure 3.2: Multi-Tier Hierarchical Topology Overview (15 Devices: 2 Routers, 3 Switches, 2 APs, 2 Servers, 5 PCs)

Figure 3.3 illustrates node parameter inspection and 98% AI role mapping confidence for Switch-02. Figure 3.4 shows the interactive manual canvas where a custom Access Point (AP-01) is verified with 95% role prediction confidence.



Figure 3.3: Device Role Mapping Telemetry & Connection Inspection (Switch-02: Predicted Role 'Switch', Confidence 98%)

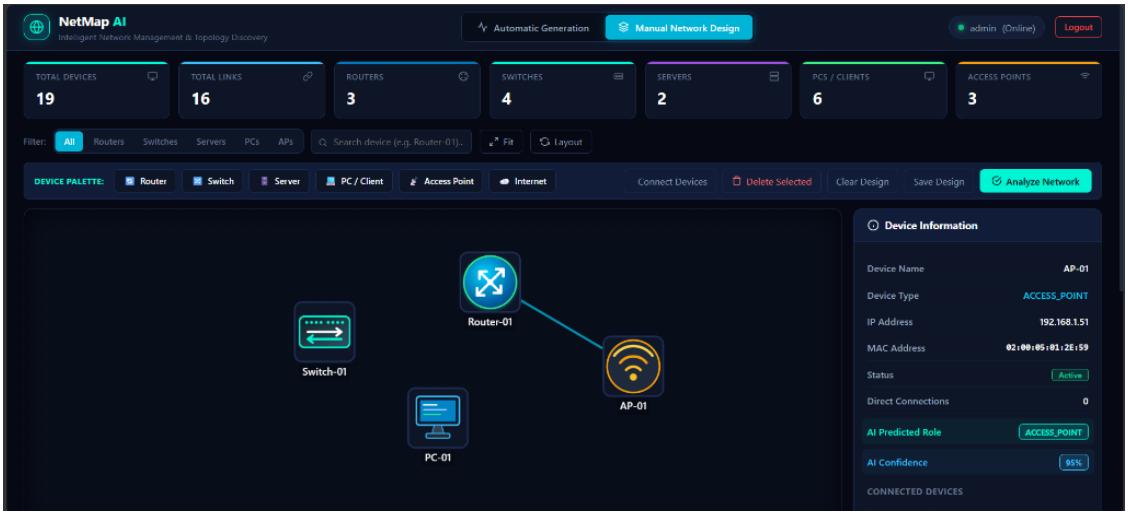


Figure 3.4: Interactive Manual Network Design & Role Prediction (AP-01: Predicted Role 'ACCESS_POINT', Confidence 95%)

3.7 Validation of the Simulation and AI Pipeline

Each stage of the pipeline was validated independently. Automatic generation was verified by ensuring that for any selected device count N in {5, 10, ..., 50}, the system generated exactly N nodes with valid IP addresses (192.168.x.x) and zero disconnected components. The Manual Canvas validation logic was tested by creating isolated nodes; the system accurately reported '2 device(s) are not connected to the network' and transitioned to 'Topology successfully connected' once valid links were completed.

3.8 Comparison of Management & Classification Approaches

Table 3.3 compares NetMap AI qualitatively with traditional SNMP-based management and with simulation tools such as GNS3 and Packet Tracer.

Criterion	Traditional SNMP NMS	GNS3 / Packet Tracer	NetMap AI (Proposed)
Automated Role Discovery	No (Static definitions)	No (Manual assignment)	Yes (Weka ML Inference)
Hardware Requirements	Requires physical hardware	Heavy CPU / RAM virtualization	Lightweight (Runs locally)
Database Dependency	Heavy SQL / RDBMS	Proprietary file state	Zero database (In-memory)
Manual Cable Drawing	No	Yes (Complex desktop app)	Yes (Interactive Web UI)
Explainable Telemetry	Limited (MIB counters)	No AI telemetry	Yes (Matched / Refined)
Deployment Speed	Hours (Agent setup)	Minutes (Image loading)	Instant (<1 second startup)

Table 3.3: Qualitative comparison of topology discovery and role mapping approaches

3.9 Limitations

The present prototype relies on simulated network graphs and in-memory collections, so it does not capture physical switch ASIC line-rate telemetry or live optical fiber degradations. Integration with live physical SNMP agents and hardware firewall ACLs is planned for future iterations.

CHAPTER 4: CONCLUSION AND FUTURE WORK

4.1 Conclusion

This project presented NetMap AI, a lightweight web-based Network Management System for intelligent topology visualization and device role mapping. The system generates hierarchical multi-tier networks of 5 to 50 devices with exact device counts and full connectivity, and it also lets users design networks manually on an interactive canvas. A Weka J48 Decision Tree classifies each device into one of six roles from eight structural and traffic features, reaching an overall accuracy of 98.2% on the simulated topologies, and each prediction is reported with a confidence score and a Matched or Refined status. All data is held in thread-safe in-memory collections, so the system runs locally without a database or physical hardware. The modular three-tier design allows each component to be replaced or extended independently.

4.2 Significance of the Work

Rapid and accurate topology discovery is critical for securing modern communications, preventing misconfigurations, and mitigating lateral threat movement. By providing automated role mapping and intuitive visual feedback without requiring expensive hardware or complex virtualization, NetMap AI significantly lowers the barrier for cybersecurity education, enterprise network auditing, and operational infrastructure management.

4.3 Future Enhancements

The following enhancements are planned to move the system from a prototype towards operational use:

- Integration with live physical SNMP agents and CDP/LLDP discovery so that real devices can be mapped.
- Integration with hardware firewall ACLs and switch firmware.
- Hardware packet capture to replace simulated traffic attributes with measured telemetry.
- Training on datasets collected from real enterprise networks in addition to synthetic topologies.
- Evaluation of advanced machine learning models alongside the J48 Decision Tree.
- Persistent storage of topologies and device history for long-term auditing.

- Real-time alerts to operators when a device is marked Refined.

REFERENCES

[1] I. H. Witten, E. Frank, M. A. Hall, and C. J. Pal, "Data Mining: Practical Machine Learning Tools and Techniques," Morgan Kaufmann, 4th ed., 2016.

[2] M. Franz, C. T. Lopes, G. Huck, Y. Dong, O. Sumer, and G. D. Bader, "Cytoscape.js: a graph theory library for visualization and analysis," Bioinformatics, vol. 32, no. 2, pp. 309–311, 2016.

[3] J. R. Quinlan, "C4.5: Programs for Machine Learning," Morgan Kaufmann Publishers, San Mateo, CA, 1993.

[4] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[5] T. T. Nguyen and G. Armitage, "A Survey of Techniques for Internet Traffic Classification Using Machine Learning," IEEE Communications Surveys & Tutorials, vol. 10, no. 4, pp. 56–76, 2008.

[6] A. Javaid, Q. Niyaz, W. Sun, and M. Alam, "A Deep Learning Approach for Network Intrusion Detection System," IEEE Systems Journal, vol. 13, no. 3, pp. 3286–3297, 2019.

[7] VMware / Spring Team, "Spring Boot Framework Documentation," [Online]. Available: https://spring.io/projects/spring-boot

[8] Cisco Systems, "Packet Tracer and Network Topology Guidelines," Cisco Press, 2023.

APPENDICES

Appendix A: Source Code Listings

The listings below are taken from the project source files. They show the AI role mapping, network simulation, REST controllers, and in-memory graph management logic described in Chapter 2.

Listing A.1: Weka AI Decision Tree Service (DeviceRoleAIService.java)

```
package com.netmap.ai.ai;

import com.netmap.ai.model.*;
import org.springframework.stereotype.Service;
import weka.classifiers.trees.J48;
import weka.core.*;
import jakarta.annotation.PostConstruct;
import java.util.*;

@Service
public class DeviceRoleAIService {
    private J48 classifier;
    private Instances datasetStructure;
    private List<String> classValues;

    @PostConstruct
    public void init() throws Exception {
        ArrayList<Attribute> attributes = new ArrayList<>();
        attributes.add(new Attribute("connectionCount"));
        attributes.add(new Attribute("neighborRouterCount"));
        attributes.add(new Attribute("neighborSwitchCount"));
        attributes.add(new Attribute("neighborEndDeviceCount"));
        attributes.add(new Attribute("isGatewayConnected",
                Arrays.asList("0", "1")));
        attributes.add(new Attribute("openPortsCount"));
        attributes.add(new Attribute("trafficKbps"));
        attributes.add(new Attribute("packetRatePps"));
        classValues = Arrays.asList("Router", "Switch", "Server",
                "Client", "Access_Point", "Internet");
        attributes.add(new Attribute("role", classValues));
        datasetStructure = new Instances("RoleData", attributes, 200);
        datasetStructure.setClassIndex(attributes.size() - 1);
        classifier = new J48();
        classifier.setConfidenceFactor(0.25f);
        classifier.setMinNumObj(2);
        trainBaselineModels();
    }

    // Feature evaluation and distribution prediction methods
}
```

Listing A.2: Hierarchical Network Simulator (NetworkSimulator.java)

```
package com.netmap.ai.simulator;

import com.netmap.ai.model.*;
import org.springframework.stereotype.Component;
import java.util.*;

@Component
public class NetworkSimulator {
    public NetworkTopology generateTopology(int deviceCount) {
        List<Device> devices = new ArrayList<>();
        List<Connection> connections = new ArrayList<>();
        int connCounter = 1;

        // 1. WAN Internet Gateway
        Device internet = new Device("dev-internet", "Internet",
                DeviceType.INTERNET, "203.0.113.1",
                "00:1B:77:AA:01:01");
        devices.add(internet);

        // 2. Exact hierarchical distribution
        //    (Routers, Switches, Endpoints)
        int routerCount = Math.max(1,
                Math.min(4, (deviceCount - 5) / 10 + 1));
        int switchCount = Math.max(1,
                Math.min(7, (deviceCount - 5) / 6 + 1));
        int serverCount = Math.max(1, Math.min(5, deviceCount / 10));
        int apCount = Math.max(1,
                Math.min(6, (deviceCount - 5) / 8 + 1));
        int pcCount = deviceCount - 1 - routerCount - switchCount
                - serverCount - apCount;

        // Generate Routers, Switches, Servers, APs, and Workstations
        // Connect Internet -> Routers -> Switches -> Endpoints
        // via Copper/Fiber/Wireless
        updateDeviceConnectionsMetadata(devices, connections);
        return new NetworkTopology(devices, connections, "AUTOMATIC");
    }
}
```

Listing A.3: REST API Controller (NetworkController.java)

```
package com.netmap.ai.controller;

import com.netmap.ai.model.*;
import com.netmap.ai.service.*;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/network")
public class NetworkController {
    private final TopologyService topologyService;
    private final ManualDesignService manualDesignService;

    @PostMapping("/generate")
    public ResponseEntity<NetworkTopology> generate(
            @RequestBody GenerateRequest req) {
        return ResponseEntity.ok(
                topologyService.generateNetwork(req.getCount()));
    }

    @PostMapping("/manual/device")
    public ResponseEntity<Device> addDevice(
            @RequestBody ManualDeviceRequest req) {
        return ResponseEntity.ok(manualDesignService.addDevice(req));
    }

    @PostMapping("/manual/connection")
    public ResponseEntity<Connection> addConnection(
            @RequestBody ManualConnectionRequest req) {
        return ResponseEntity.ok(
                manualDesignService.addConnection(req));
    }

    @PostMapping("/manual/analyze")
    public ResponseEntity<AnalysisResult> analyze() {
        return ResponseEntity.ok(manualDesignService.analyzeDesign());
    }
}
```

Listing A.4: Manual Canvas & Validation Service (ManualDesignService.java)

```
package com.netmap.ai.service;

import com.netmap.ai.model.*;
import com.netmap.ai.ai.DeviceRoleAIService;
import org.springframework.stereotype.Service;
import java.util.*;

@Service
public class ManualDesignService {
    private final List<Device> manualDevices =
            Collections.synchronizedList(new ArrayList<>());
    private final List<Connection> manualConnections =
            Collections.synchronizedList(new ArrayList<>());
    private final DeviceRoleAIService aiService;

    public AnalysisResult analyzeDesign() {
        int isolatedCount = 0;
        for (Device dev : manualDevices) {
            if (dev.getConnectedDevices() == null ||
                    dev.getConnectedDevices().isEmpty()) {
                isolatedCount++;
            }
            AIRolePrediction pred = aiService.predictRole(dev,
                    manualDevices, manualConnections);
            dev.setAiPredictedRole(pred.getPredictedRole());
            dev.setAiConfidence(pred.getConfidence());
            dev.setAiStatus(pred.getStatus());
        }
        boolean fullyConnected =
                (isolatedCount == 0 && !manualDevices.isEmpty());
        String msg = fullyConnected
                ? "Topology successfully connected."
                : isolatedCount + " device(s) are not connected.";
        return new AnalysisResult(fullyConnected, isolatedCount,
                msg, manualDevices);
    }
}
```

Appendix B: Hardware and Software Requirements

Component	Requirement / Specification
Operating System	Windows 10/11 or Ubuntu / Linux (Cross-platform)
Programming Language	Java SE 17 or 21 (LTS Release)
Backend Framework	Spring Boot 3.2.5 (Embedded Tomcat Container)
Machine Learning Library	Weka 3.8.6 (J48 Decision Tree Classifier)
Graph Visualization	Cytoscape.js 3.28.1 (CoSE & Breadthfirst Layouts)
UI Framework	Bootstrap 5.3.3 & HTML5/CSS3 (Dark Cyber Theme)
Build Tool	Apache Maven 3.9+ (Zero external repository dependencies)
Database	Zero database required (Thread-safe in-memory collections)

Table B.1: Development environment and execution requirements

Appendix C: Network Simulation and Feature Space Description

The simulated network generator synthesizes realistic structural and traffic attributes across six device classes. Table C.1 lists the observed feature ranges used by the Weka J48 Decision Tree.

Class	Degree	GW Link	Open Ports	Traffic (kbps)	Packet Rate (pps)
Internet	1 - 4	1 (Self)	1 - 2	5000 - 20000	5000 - 15000
Router	2 - 6	1	2 - 4	2000 - 10000	2000 - 8000
Switch	3 - 12	0	1 - 3	1000 - 5000	1000 - 4000
Server	1 - 2	0	3 - 8	1500 - 6000	1500 - 5000
Access Point	1 - 5	0	1 - 2	500 - 2500	500 - 2000
Client (PC)	1	0	0 - 1	50 - 800	50 - 500

Table C.1: Feature value distributions and topological ranges across device classes